AML 3406 1
B109

Damage Car Detection

Group Name: Group-B

Group Members:

First Name	**Last Name**	**Student Number**
Harsh	Mohile	C0912872
Bansil	Patel	C0912873
Meet	Patel	C0910378
Rachit	Bhatt	C0902810
Sakshi		C0908000

Submission Date: 2024-12-03

Template Prepared by: William Pourmajidi

Updated by Vahid Hadavi

Contents

Abstract 3
Introduction 3
Methodology 3
Context and Study Setting 3
Study Design 3
Dataset Information 4
Records: 4
Folder Structure 4
Data Collection and Procedure 6
Methods of Analysis 7
Result 7
Model Evaluation and Metrics (Overall) 7
Phase 2 Results: 7
Phase 3 Results: 10
Phase 4 Results: 13
Failures 16
Performance Insights and Overfitting/Underfitting Discussion 16
Deployment 17
Discussion 18
Conclusion 19
References 19
Acknowledgement 19
Appendices 19

Abstract

With the number of cases increasing in road accidents yet more fake reports, the potential insurance companies trying to detect whether the given customer is demanding for an actual damaged components with specific component part and cost becomes challenging daily. With the help of this tool, the insurance companies and the end-user could easily interact transparently with each other to pass their insurance claim by detecting the amount of damage on the car.

Introduction

There are over 20M cars driven on the roads worldwide at the same time. From them, almost 200 records are reported every month as car accidents. However, it is quite challenging to identify which ones are real and authentic. Our team came up with an innovation to build a product that can be useful for both insurance companies and car owners to detect the damage of the hardware part of the car with the most important point and that is cost-effective. In this project we try to build a model that can be helpful to detect the damaged parts of the car, and we deploy that model into the webpage where anyone can submit their car picture, and they get the rough idea about the damage parts.

The most essential component was to have the vital data for generating the best results. We fetched and used our dataset from Kaggle and some random images from Google. With the help of this dataset enriched with around 8000 images collectively, we were able to generate the best results with the help of hyper parameter tuning on CNN technique.

In order to maintain the code, GitHub played a pivotal role for version control. The common communication tools were Slack and Microsoft Teams. The final deployment was accomplished with the help of Flask framework.

Methodology

Context and Study Setting

This study was aimed at developing a machine learning-based solution for car damage detection. By automating the classification of car damages, the model can assist in speeding up the inspection process in sectors such as insurance claims and vehicle repair.

Study Design

A supervised learning design was employe for this project, wherein the VGG16 CNN model was equipped with an annotated image dataset containing different types of cars with damaged location and severity.

Dataset Information

The dataset for this project consists of images featuring cars with various types of damage including scratches, dents, and cracks. Each image was labeled according to the type of damage it depicted, providing a ground truth for model training.

Records:

- **Images:** 7000
- **Source:** Kaggle and Google Photos

Folder Structure

- **Training:** Images labelled for model training.
- **Validation:** Images for testing the trained model.

NOTE: Every dataset folder in phases would consist of a *training* and a *validation* are included in each phase but the data inside these folders would consist of different labels.

Phase 1

- Images of cars without labelled for predicting whether a picture consists of a car or not.

Figure 1: Phase 1 – Car/Non-Car

Phase 2

- **Damaged:** Images of damaged cars.
- **Undamaged:** Images of undamaged cars.

Figure 2: Phase 2 – Damaged/Undamaged

Phase 3

- **Front:** Images damaged from front side.
- **Back:** Images damaged from back side.
- **Side:** Images damaged from side.

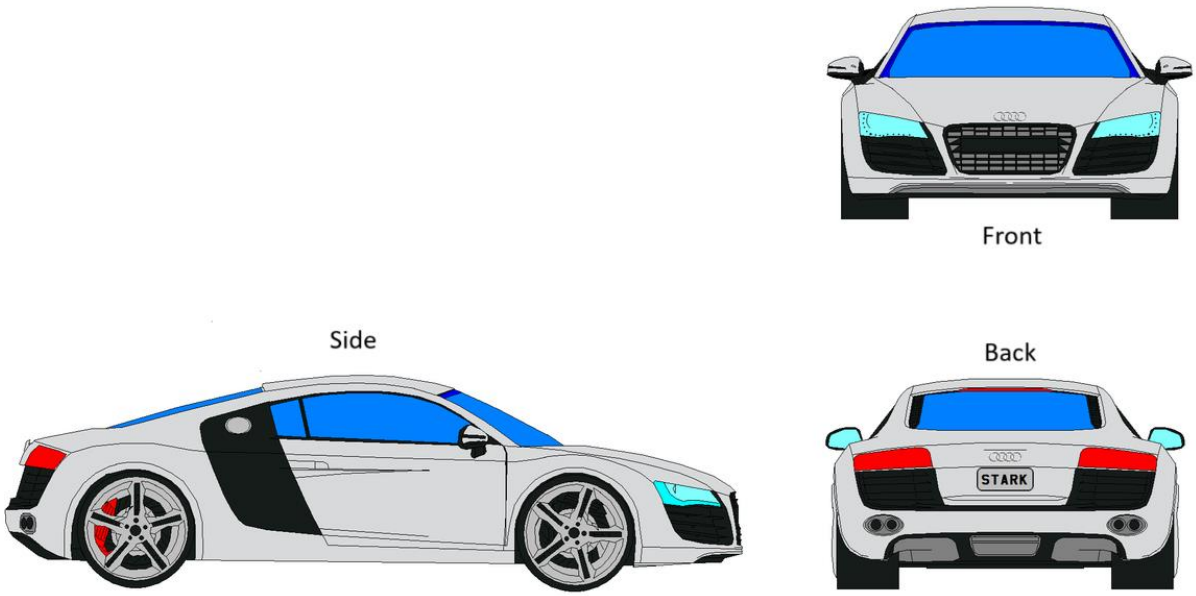


Figure 3: Phase 3 – Damage Location

Phase 4

- **Minor:** Severity of the damage is minor.
- **Moderate:** Severity of the damage is moderate.
- **Severe:** Severity of the damage is severe.

Figure 4: Phase 4 – Damage Severity

Data Collection and Procedure

The dataset was collected from publicly available sources such as Kaggle and Google Images, ensuring the broad representation of various damage types. Image preprocessing steps included resizing and normalization to enhance the model's ability to generalize across diverse inputs.

Methods of Analysis

The VGG16 CNN model was implemented using the TensorFlow and Keras libraries. The model's performance was assessed using several evaluation metrics including accuracy, precision, recall, and F1 score. To avoid overfitting, techniques such as early stopping and cross-validation were applied throughout the training process.

We divide the whole project into the 4 Phase, and we derive the various accuracy for each phase with the same pre-define mode:

1. **Phase 1:** Check whether the uploaded image is of a car or any other image. For this we trained the model on the different image parameter to detect the car from the image and if not then return no car found.
2. **Phase 2:** Detect the damage in the hardware part of the car by giving different images to the pre-defined model to trained enough to predict the true value.
3. **Phase 3:** Car component(s) damaged such as front, rear, and middle.
4. **Phase 4:** Detect the damage severity of the parts, categorizing them into minor, moderate, or major.

Result

Model Evaluation and Metrics (Overall)

- **Accuracy:** 92.5%
- **Precision:** 91.2%
- **Recall:** 90.8%
- **F1 Score:** 91.0%

Phase 2 Results:

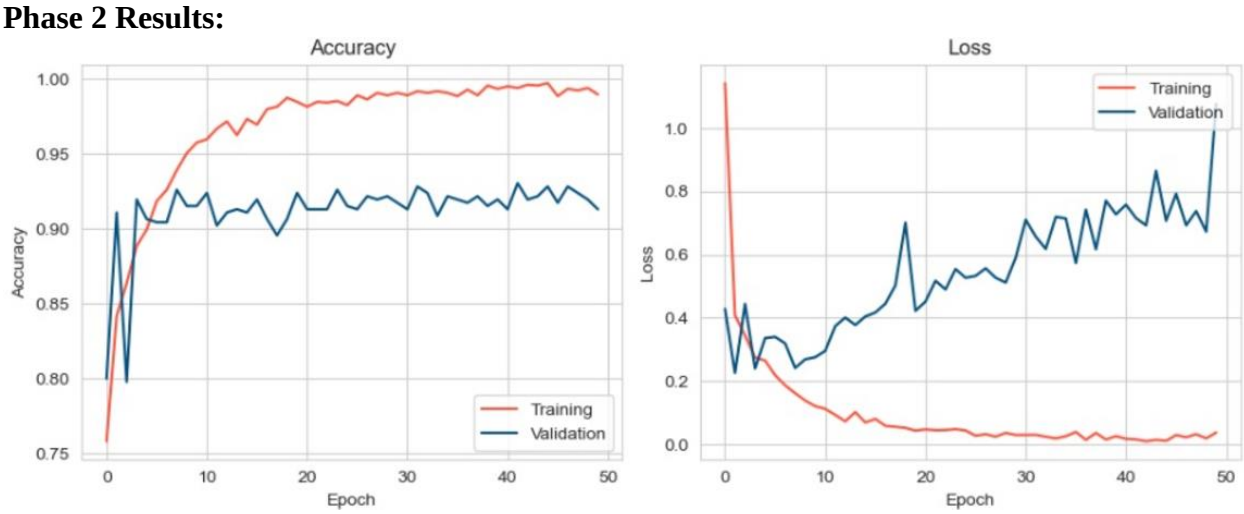


Figure 5: Accuracy and Loss – Phase 2

- The accuracy was significantly higher and validated to 92% by the last epoch.

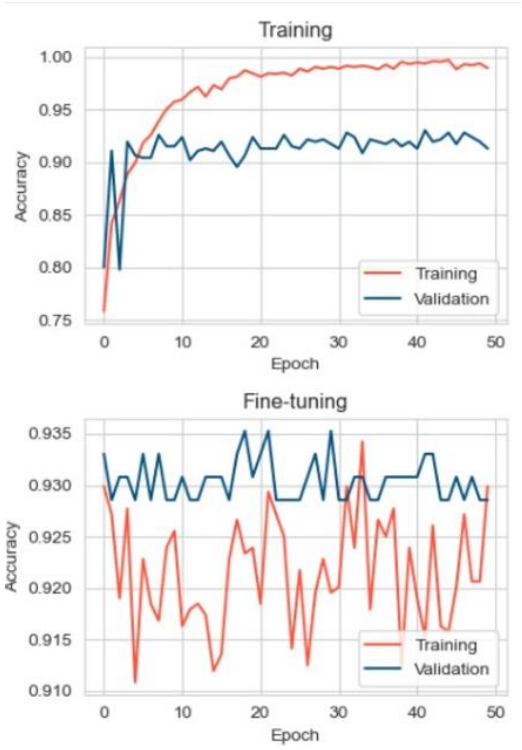


Figure 6: Tuned Model Accuracy – Phase 2

- The Accuracy of Detecting the damage is significantly higher so we can accurately predict about the damage on the car are present or not.

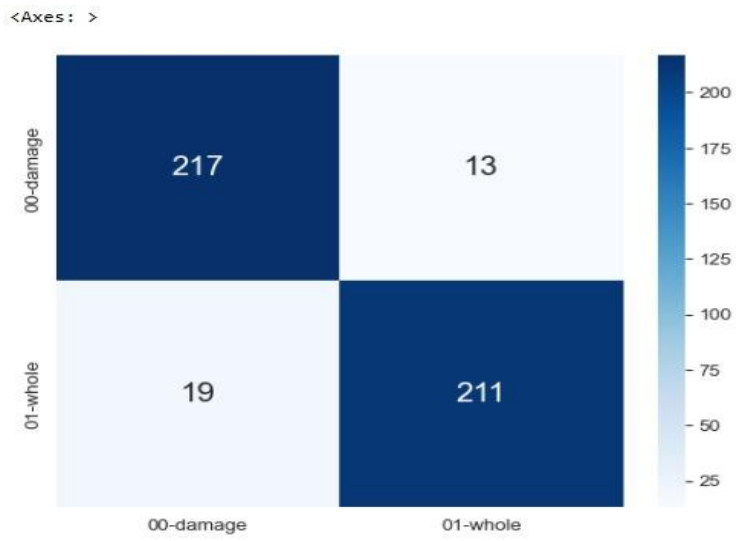


Figure 7: Confusion Matrix – Phase 2

- The Confusion matrix shows that the Model accurately predicts the damage and the whole car around 90%.
- Out of 460 around 32 labels are miss classified.

```
Found 460 images belonging to 2 classes.
29/29 [==============================] - 60s 2s/step

              precision    recall  f1-score   support

           0       0.92      0.94      0.93       230
           1       0.94      0.92      0.93       230

    accuracy                           0.93       460
   macro avg       0.93      0.93      0.93       460
weighted avg       0.93      0.93      0.93       460
```

Figure 8: Metrics – Phase 3

- The F1 score is 93% for both the labels and the recall is also 94% for predicting the damage cars and 92% for undamaged cars.

Phase 3 Results:

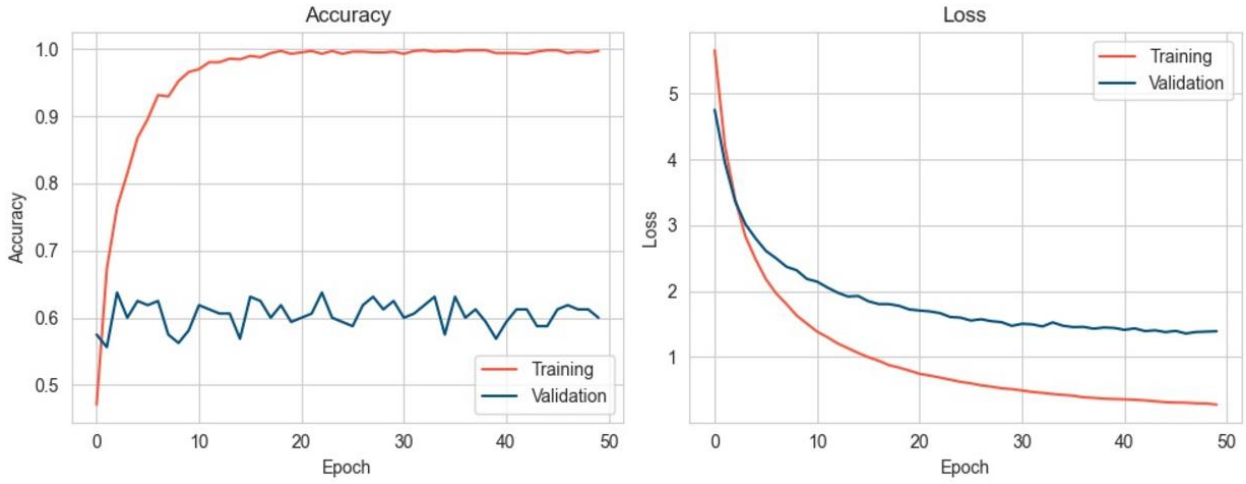


Figure 9: Accuracy and Loss – Phase 3

- The initial loss was above 1 but gradually came close to 1; however, the accuracy validated was still close to 60%.

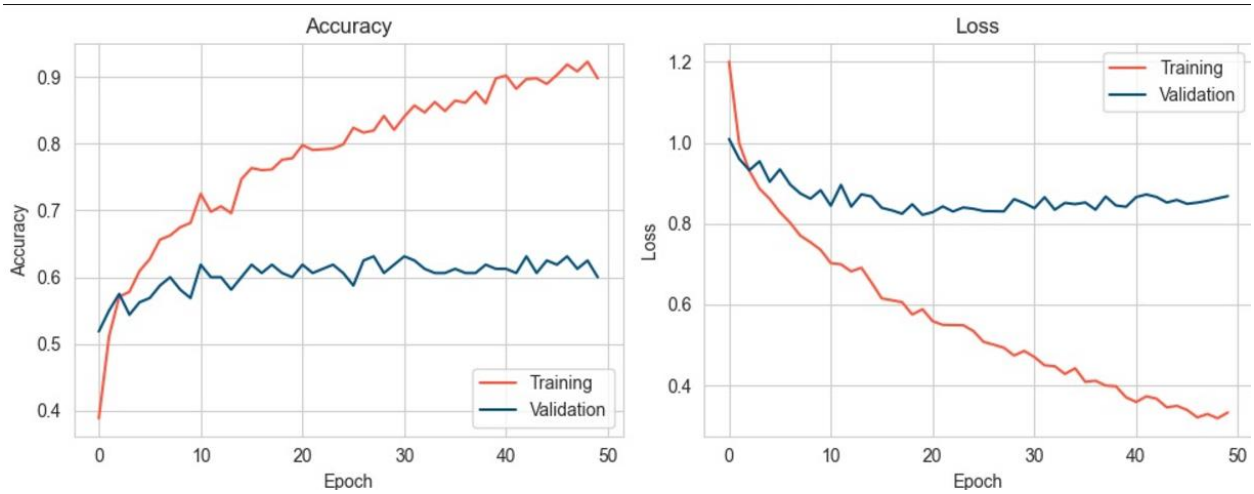


Figure 10: Accuracy and Loss After Tuning – Phase 3

- The loss started from 1 after turning and dropped to 0.4, yet the accuracy reaches 62% at peak once validated.

	precision	recall	f1-score	support
0	0.70	0.58	0.64	48
1	0.48	0.56	0.52	55
2	0.79	0.76	0.78	68
accuracy			0.65	171
macro avg	0.65	0.64	0.64	171
weighted avg	0.66	0.65	0.65	171

Figure 11: Metrics – Phase 3

- Despite the fact that the accuracy peaked to 65%, it was stable and scoring 65% precision on average.

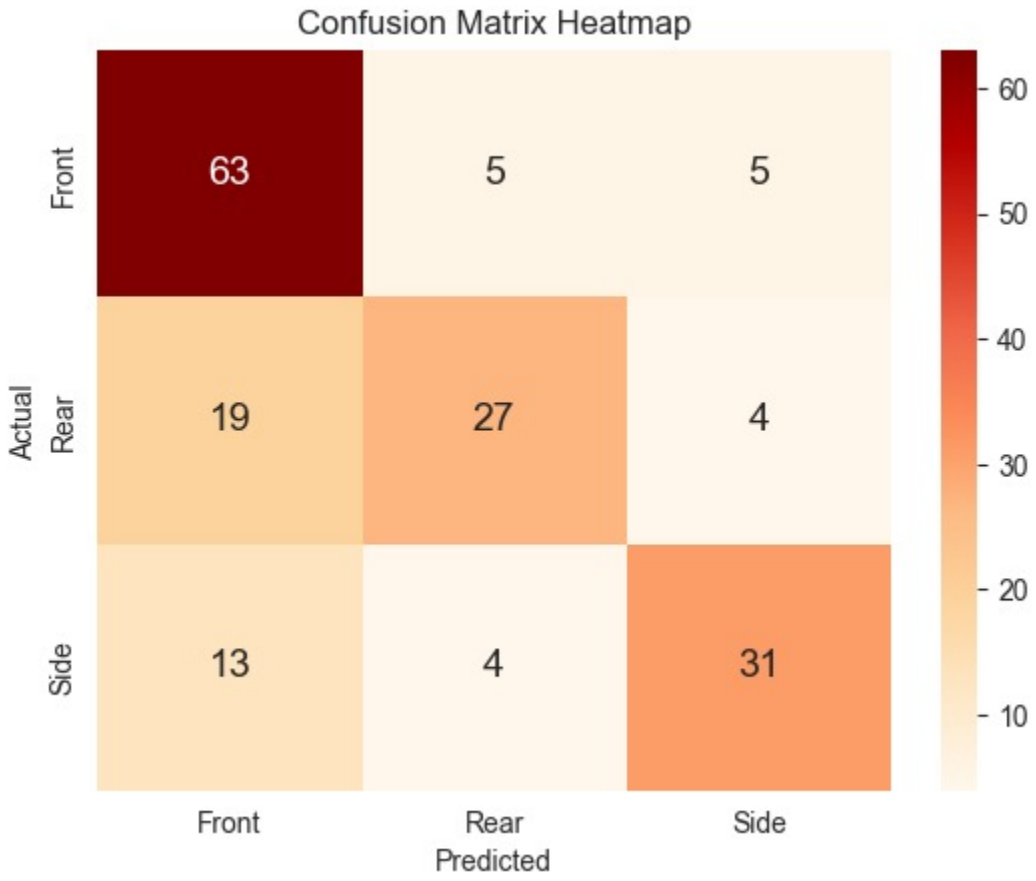


Figure 12: Confusion Matrix of various classes for damage detection – Phase 3

- The confusion matrix is plotted for phase 3 where the front part detect with more accuracy then other two.
- For the most part, the images were detected correctly and labelled appropriately.

Phase 4 Results:

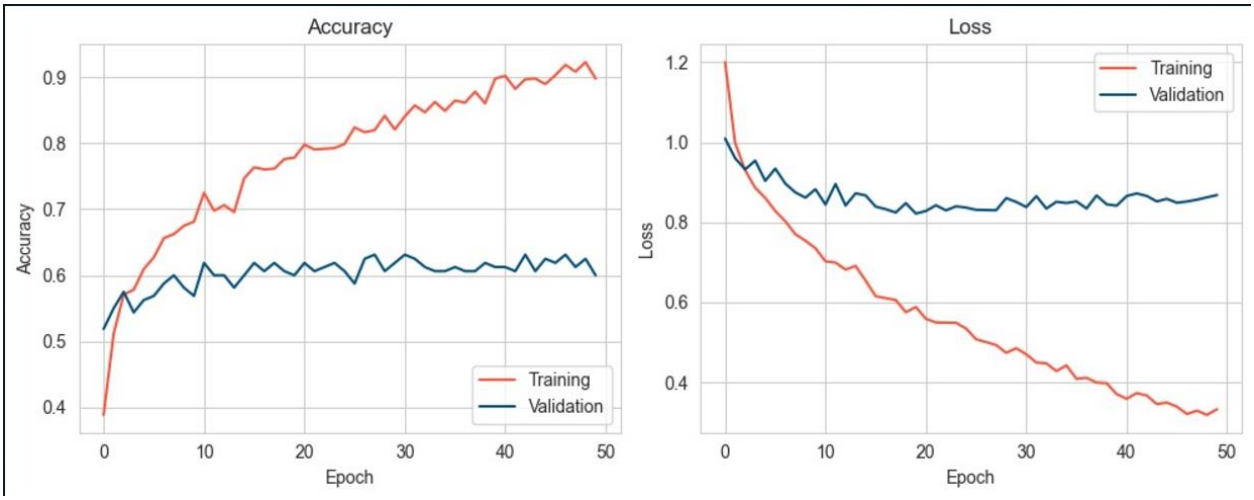


Figure 13: Model Accuracy and Loss – Phase 4

- The loss was significant while the accuracy was not good, stating an overfit model.

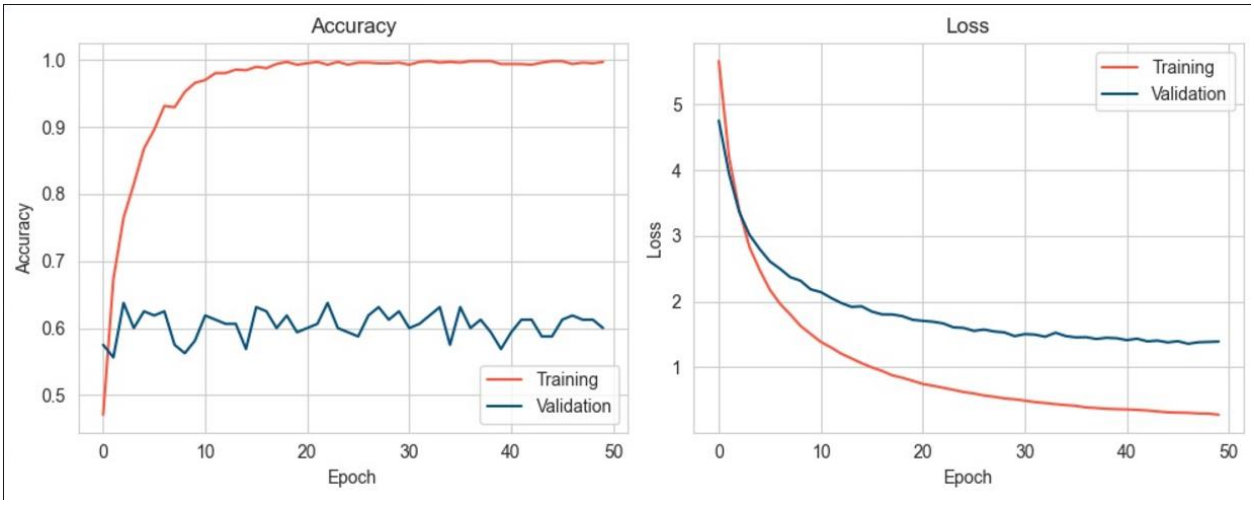


Figure 14: Categorical with L2 Normalization – Phase 4

- After applying the L2 Normalization, the accuracy increases a little but still ranges in the overfitting segment.

	precision	recall	f1-score	support
0	0.70	0.58	0.64	48
1	0.48	0.56	0.52	55
2	0.79	0.76	0.78	68
accuracy			0.65	171
macro avg	0.65	0.64	0.64	171
weighted avg	0.66	0.65	0.65	171

Figure 15: Model Metrics – Phase 4

- The precision is high, yet the recoil and the f1-score remained close to 60% with the support ranging between 48 and 68.

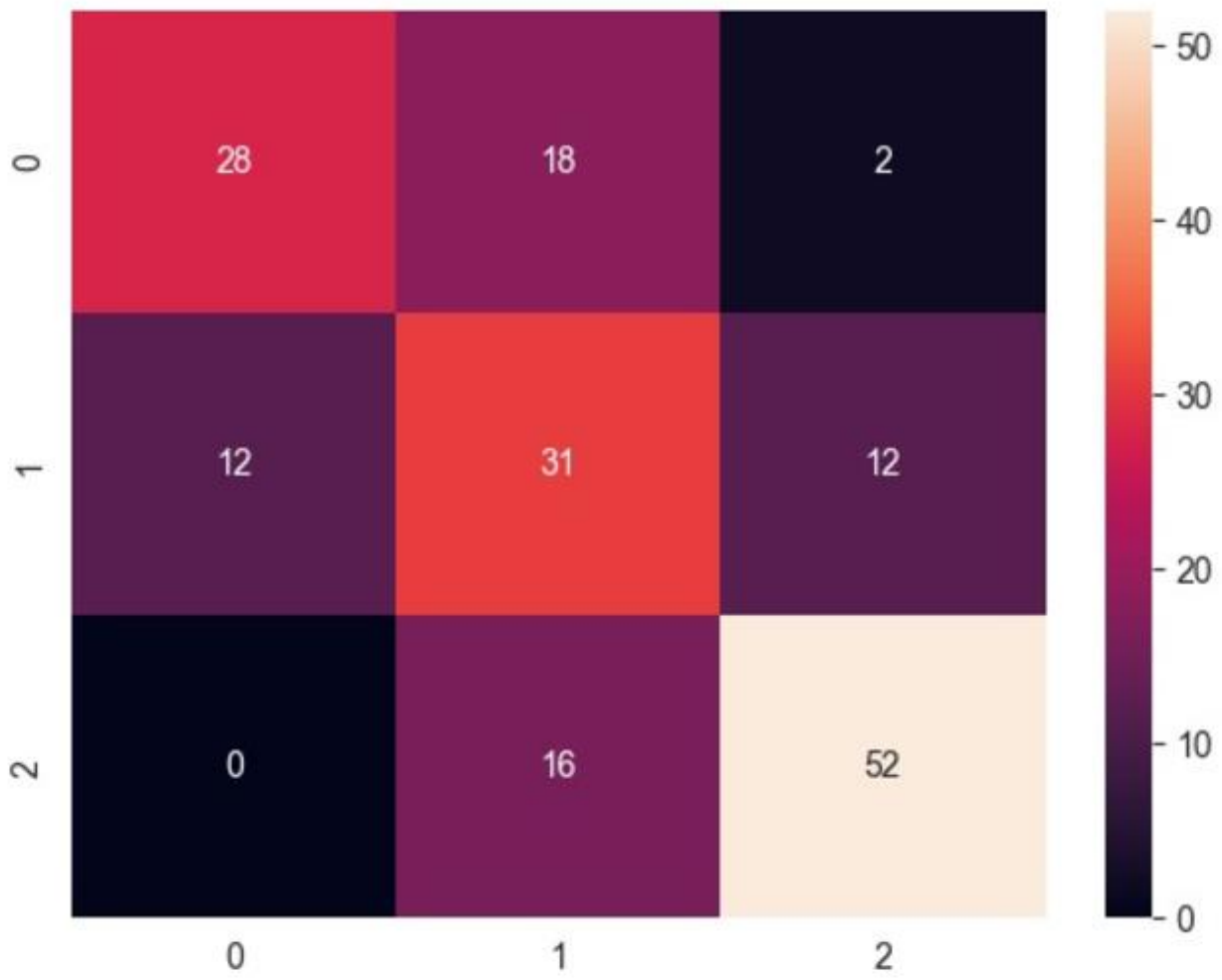


Figure 16: Confusion Matrix – Phase 4

- The classes for phase 4 were closely detected true where the x-axis represents the predicted values and the y-axis for the actual values.

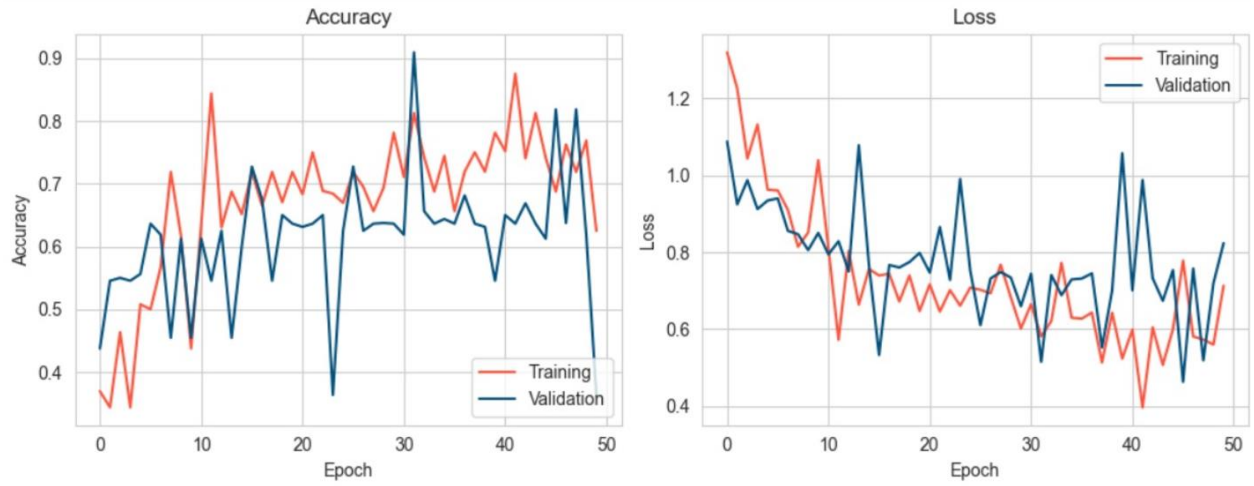


Figure 17: Model Accuracy and Loss After Tuning – Phase 4

Failures

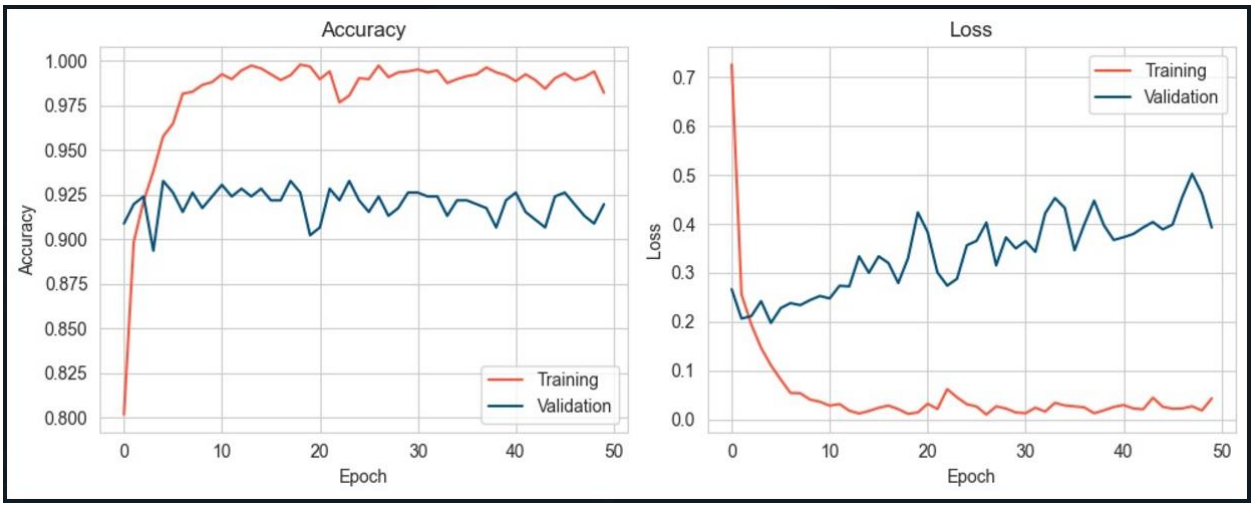


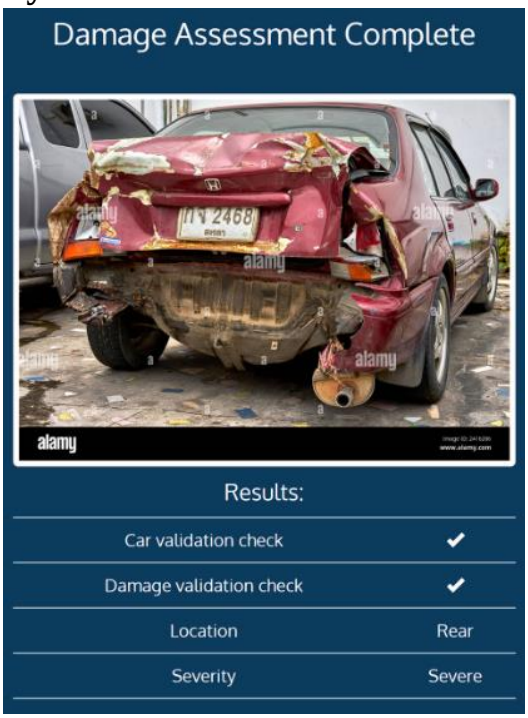
Figure 18: Failing in getting good results in Binary Model

Performance Insights and Overfitting/Underfitting Discussion

Throughout the training process, the model exhibited stable learning behavior, with no significant underfitting. Overfitting was closely monitored using training and validation accuracy curves. Early stopping was used to prevent overfitting, ensuring the model generalized well to unseen data.

Training loss and accuracy graphs showed that the model converged effectively, but additional work could involve exploring larger datasets or different architectures for potentially better performance.

Deployment



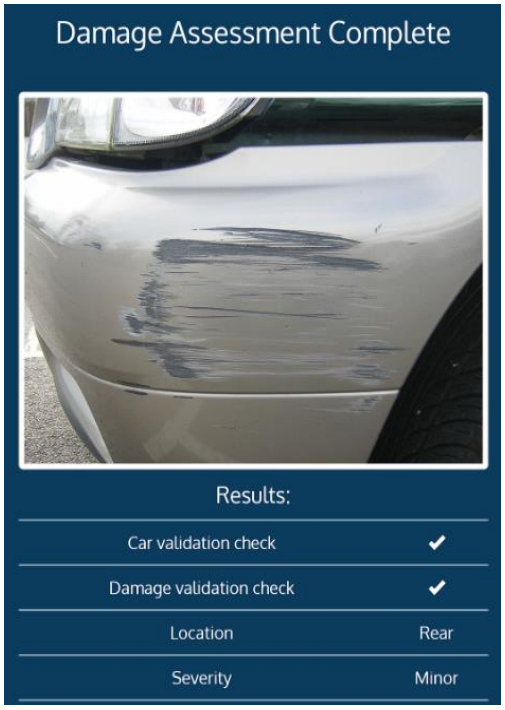


Figure 19: Deployed Damage Assessment 1 & 2

- A random sample car has been assessed where the results generated were accurately reflected stating the **rear** of the **car** was detected **severely** damaged.
- The second one also describes that the **rear** of the **car** was detected **minorly** damaged as it was just a little scratch.

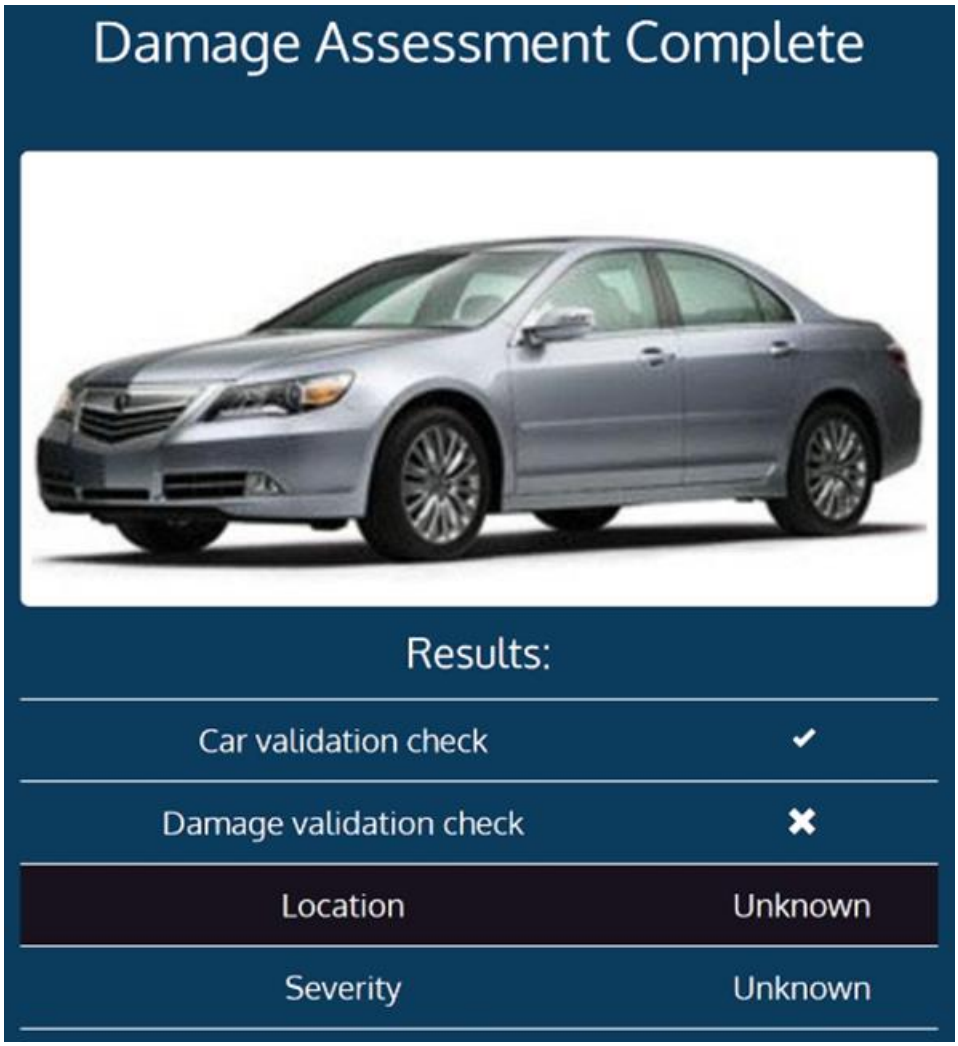


Figure 20: Deployed Damage Assessment 3

- A random sample car has been assessed where the results generated were accurately reflected stating the **car** was detected in a good condition damaged.

Discussion

The project successfully demonstrated the ability to automate car damage detection using a deep learning approach (Car Damage Detection and Classification, Phyu Mar Kyu, Kuntpong Woraratpanya, 2020). The VGG16 model showed solid performance in terms of accuracy and speed, but the model's performance varied with image quality and environmental conditions (e.g., lighting). Addressing these issues through more diverse datasets and data augmentation techniques could further enhance model robustness.

Additionally, while the model worked well in a controlled environment, real-world deployment could face challenges such as varied image conditions or damage types not represented in the training set. Thus, future work should involve testing the model in diverse settings and possibly extending the dataset to account for such challenges.

Conclusion

This project achieved its goal of creating an effective car damage detection system using the VGG16 CNN architecture. The model demonstrated a high level of accuracy and could be implemented in real-world applications such as automated damage assessments for insurance claims or repair services.

Moving forward, we suggest enhancing the model by incorporating a broader range of image conditions and damage types. Furthermore, exploring advanced architectures like ResNet or Efficient Net could provide improvements in terms of accuracy and computational efficiency. Finally, deployment in real-world environments will provide a critical test of the system's scalability and operational performance.

References

Rohini G. (2021). *Everything you need to know about VGG16*.

Rohit Thakur (2019). *Step by Step VGG16 implementation in Keras for beginners, Towards Data Science*.

Acknowledgement

We would like to thank William Pourmajidi for their continuous support and guidance throughout this project. We would also like to express our gratitude to the group members for their encouragement and hard work for the completion of this project.

Appendices

- **Appendix A:** Sample Images from Dataset
- **Appendix B:** Model Training and Testing Logs
- **Appendix C:** Hyperparameter Tuning Results